# ANALISI OOP

## Space Invaders

*Esercizi di analisi orientata agli oggetti in C++*

## Come usare questo documento

Questo documento ti guida nell'analisi del gioco Space Invaders usando i concetti della Programmazione Orientata agli Oggetti (OOP) che hai studiato. Non si tratta di scrivere il codice completo del gioco, ma di allenare il tuo pensiero da progettista: capire quali classi servono, cosa devono sapere e cosa devono fare.

Per ogni sezione troverai:

- Una breve descrizione della competenza OOP affrontata
- Il modello delle classi di Space Invaders come punto di riferimento
- Domande aperte da rispondere con ragionamenti motivati
- Una traccia (hint) per orientarti se sei in difficoltà

Attenzione alle domande trabocchetto (segnalate con il simbolo ⚠): sono domande con una risposta intuitiva ma errata. Scopri perché!

## Space Invaders in poche parole

*Leggi attentamente: da questa descrizione nascono le classi*

Space Invaders è un gioco arcade spaziale a schermo fisso in cui il giocatore controlla una navicella che si sposta orizzontalmente nella parte bassa dello schermo. Lo scopo è distruggere tutti gli alieni presenti sullo schermo sparando proiettili verso l'alto, senza essere colpito dai proiettili nemici né dalla discesa degli alieni stessi.

Gli alieni sono disposti in una formazione a griglia (tipicamente 11 colonne per 5 righe) e si muovono in blocco orizzontalmente: quando l'alieno più esterno raggiunge il bordo dello schermo, l'intera formazione scende di una riga e inverte la direzione. Man mano che gli alieni vengono eliminati, la formazione rimanente accelera. Se un alieno raggiunge la parte bassa dello schermo, la partita termina.

Il giocatore può ripararsi dietro quattro barriere che si erodono progressivamente, assorbendo sia i proiettili nemici sia quelli del giocatore. Periodicamente appare un UFO (nave misteriosa) nella parte alta dello schermo che si muove orizzontalmente: distruggerlo vale un punteggio variabile.

### Obiettivo

Pilotare la navicella spaziale orizzontalmente nella parte inferiore dello schermo, distruggendo tutti gli alieni presenti sparando proiettili verso l'alto, evitando i proiettili nemici e impedendo che gli alieni raggiungano il basso dello schermo. Usare strategicamente le barriere come difesa e cercare di abbattere l'UFO per ottenere punti bonus.

## Elementi di Space Invaders

| Oggetto | Punti | Note |
|---|---|---|
| Alieno di tipo A (basso) | 10 pt | Prima riga degli alieni; distrutto da un proiettile |
| Alieno di tipo B (medio) | 20 pt | Righe centrali degli alieni; distrutto da un proiettile |
| Alieno di tipo C (alto) | 30 pt | Righe superiori degli alieni; distrutto da un proiettile |
| UFO / Nave misteriosa | da 50 a 300 pt | Appare periodicamente in cima allo schermo; valore variabile |

## Azioni

- Muovi a sinistra / Muovi a destra – sposta la navicella orizzontalmente senza modificarne l'altezza.
- Spara – lancia un proiettile verticalmente verso l'alto dalla posizione attuale della navicella; il proiettile viaggia in linea retta e scompare al primo contatto con un alieno, una barriera o il bordo superiore.
- (Limitazione) – il giocatore può avere al massimo un proiettile attivo alla volta nella versione originale del gioco.

## Glossario Minimo

| Termine | Significato |
|---|---|
| Arena | Lo spazio di gioco definito da larghezza e altezza; gestisce i limiti del campo e il rilevamento dei bordi. |
| Campo di gioco | Area rettangolare a bordi fissi: gli oggetti che raggiungono i bordi laterali si fermano o vengono rimossi. |
| Navicella | Il vettore controllato dal giocatore; ha posizione, numero di vite e si muove solo orizzontalmente. |
| Alieno | Nemico che si muove in formazione orizzontale e scende a ogni cambio di direzione; esiste in tre tipi (A, B, C) con punteggi diversi. |
| Formazione | Il gruppo di alieni si muove compatto: quando l'alieno più esterno raggiunge il bordo, tutta la formazione scende di una riga e inverte la direzione. |
| Proiettile | Oggetto sparato dalla navicella o dagli alieni; viaggia in linea retta verticale e ha una durata di vita limitata. |
| Barriera | Struttura difensiva posizionata davanti alla navicella; si erode progressivamente assorbendo i proiettili nemici e del giocatore. |
| UFO | Nave misteriosa che appare periodicamente nella parte alta dello schermo e assegna un punteggio variabile se distrutta. |
| Collisione | Contatto tra due oggetti che provoca la distruzione di uno o entrambi (es. proiettile-alieno) o l'erosione (es. proiettile-barriera). |

# Responsabilità delle Classi

Ogni classe ha una responsabilità precisa. La tabella mostra cosa contiene e cosa sa fare ciascuna.

| Classe | Contiene | Sa fare |
|---|---|---|
| Arena | Larghezza Altezza | Definire i limiti del campo di gioco Rilevare quando un oggetto raggiunge i bordi |
| OggSpaziale | Posizione (x, y) Velocità (Vx, Vy) Raggio come dimensione Stato: attivo / distrutto | Dire se due oggetti sono in collisione Aggiornare la posizione Rilevare la propria distruzione Dire il proprio raggio di collisione Visualizzare se stesso sullo schermo |
| Navicella | Numero di vite rimanenti Stato: vulnerabile / invulnerabile OggSpaziale | Spostarsi a sinistra e a destra Generare un Proiettile |
| Alieno | Tipo: A / B / C OggSpaziale | Dire il proprio tipo Spostarsi orizzontalmente e scendere Generare un Proiettile nemico |
| Proiettile | Timer di vita residua OggSpaziale | Sapere se è ancora attivo (timer > 0) |
| Barriera | Punti vita (resistenza) OggSpaziale | Assorbire un proiettile riducendo la propria resistenza Rilevarsi completamente distrutta |
| UFO | Timer di comparsa OggSpaziale | Apparire periodicamente Assegnare punteggio variabile quando distrutto |
| GameEngine | Arena Navicella Array di Alieni Array di Proiettili Array di Barriere UFO Punteggio Stato del gioco: livello, esito | Avviare un nuovo livello Aggiornare tutte le entità a ogni frame Rilevare e risolvere tutte le collisioni Determinare la fine del livello o della partita |
| VistaTestuale | Campo Tastiera LettoreInput | Disegnare tutte le entità a schermo Leggere i comandi del giocatore Mostrare punteggio, vite e messaggi di esito |

# Relazioni tra le Classi

## Tabella delle relazioni

Due tipi di relazione:

- Possiede: l'oggetto B fa parte di A e viene distrutto insieme a lui.
- Usa: A ha temporaneamente bisogno di B per svolgere un'operazione.

| Chi | Relazione | Con chi | Nota |
|---|---|---|---|
| Navicella | Possiede e Usa | OggSpaziale | Navicella possiede il proprio OggSpaziale: posizione, velocità e |

| Chi | Relazione | Con chi | Nota |
|---|---|---|---|
| | | | raggio. Lo usa per spostarsi e rilevare collisioni. |
| Navicella | Possiede e Usa | Proiettile | Navicella genera un Proiettile quando spara; il proiettile viene passato a GameEngine che lo gestisce. |
| Alieno | Possiede e Usa | OggSpaziale | Alieno possiede il proprio OggSpaziale: posizione, velocità e raggio. Lo usa per muoversi e rilevare collisioni. |
| Alieno | Possiede e Usa | Proiettile | Alieno genera un Proiettile nemico; dopo lo sparo viene passato a GameEngine che lo gestisce. |
| Proiettile | Possiede e Usa | OggSpaziale | Proiettile possiede il proprio OggSpaziale: posizione e velocità. Lo usa per muoversi e rilevare collisioni. |
| Barriera | Possiede e Usa | OggSpaziale | Barriera possiede il proprio OggSpaziale: posizione e raggio. Lo usa per rilevare le collisioni con i proiettili. |
| UFO | Possiede e Usa | OggSpaziale | UFO possiede il proprio OggSpaziale: posizione e velocità. Lo usa per muoversi in cima allo schermo. |
| GameEngine | Possiede e Usa | Arena | GameEngine possiede Arena: le dimensioni del campo di gioco. La usa per calcolare le posizioni e i limiti. |
| GameEngine | Possiede e Usa | Navicella | GameEngine possiede la Navicella del giocatore ed è responsabile della sua gestione per tutta la partita. |
| GameEngine | Possiede e Usa | Alieno[] | GameEngine possiede tutti gli Alieni attivi: li crea a inizio livello e li rimuove quando vengono distrutti. |
| GameEngine | Possiede e Usa | Proiettile[] | GameEngine possiede tutti i Proiettili attivi (del giocatore e nemici) e li rimuove quando si esauriscono o colpiscono. |
| GameEngine | Possiede e Usa | Barriera[] | GameEngine possiede le Barriere e ne gestisce la progressiva erosione durante il gioco. |
| GameEngine | Possiede e Usa | UFO | GameEngine gestisce la comparsa periodica dell'UFO e la sua eventuale distruzione. |
| IOTestuale | Possiede e Usa | Campo | IOTestuale possiede un campo da gioco e lo usa per visualizzare l'output su console. |
| IOTestuale | Possiede e Usa | Tastiera | IOTestuale possiede una Tastiera e la usa per leggere i tasti direzionali premuti dal giocatore. |
| IOTestuale | Possiede e Usa | LettoreInput | IOTestuale possiede LettoreInput e lo usa per chiedere il nome del giocatore. |

| Chi | Relazione | Con chi | Nota |
|---|---|---|---|
| GameEngine | Usa | IOTestuale | GameEngine usa IOTestuale per leggere le indicazioni dell'utente e per visualizzare informazioni. |
| OggSpaziale | Usa | IOTestuale | OggSpaziale usa IOTestuale per visualizzare il suo contenuto. |

# Ciclo di un livello (5 passi)

1. **Inizio livello:** GameEngine crea l'Arena con le dimensioni della finestra, inizializza la Navicella in basso al centro e genera la formazione di Alieni nella metà superiore dello schermo. Posiziona le quattro Barriere nella zona intermedia. GameEngine usa VistaTestuale per mostrare lo stato iniziale e attendere il primo input.
2. **Ciclo di aggiornamento (game loop):** a ogni frame GameEngine legge i comandi da VistaTestuale e li applica alla Navicella (spostamento a sinistra/destra, sparo). Aggiorna poi la posizione di ogni entità attiva: la formazione di Alieni si muove in blocco orizzontalmente e scende quando raggiunge un bordo; i Proiettili (del giocatore e nemici) avanzano verticalmente.
3. **Rilevamento collisioni:** GameEngine verifica tutte le coppie rilevanti: Proiettile/Alieno, Proiettile/Barriera, ProiettileNemico/Barriera, ProiettileNemico/Navicella, Navicella/Alieno (se la formazione è scesa troppo), Proiettile/UFO. Per ogni collisione rimuove le entità coinvolte, erode le Barriere e aggiorna il punteggio.
4. **Gestione UFO e alieni:** un timer controlla la comparsa periodica dell'UFO che attraversa la parte alta dello schermo. Gli alieni superstiti sparano casualmente; GameEngine aggiunge i proiettili nemici al proprio array e li gestisce come gli altri. La formazione accelera progressivamente man mano che gli alieni vengono eliminati.
5. **Esito:** se tutti gli Alieni sono stati distrutti, il livello è completato e GameEngine genera una nuova formazione più numerosa e veloce. Se la Navicella viene colpita scala una vita e ricompare brevemente invulnerabile. Se la Navicella esaurisce le vite o un Alieno raggiunge la parte bassa dello schermo, la partita termina. In entrambi i casi VistaTestuale mostra il messaggio di esito e il punteggio finale.

# Scelte di Progetto

## Perché tre classi per l'input/output?

Nella versione attuale il gioco funziona sul terminale: Campo usa cout per disegnare le entità con caratteri ASCII, Tastiera usa cin per leggere i comandi direzionali e di fuoco. Potrebbe sembrare eccessivo usare tre classi per qualcosa di così semplice.

La ragione è la seguente: se in futuro si vorrà aggiungere la grafica con SFML, sarà necessario sostituire la stampa su terminale con il rendering di una finestra. Con questa architettura:

- Campo – definisce dove e come si disegna qualcosa (oggi: terminale ASCII, domani: finestra SFML con sprite).
- Tastiera – definisce come si legge un comando (oggi: cin, domani: sf::Event di SFML).

- LettoreInput – coordina la visualizzazione di un prompt e la raccolta della risposta.

Quando si aggiungerà la grafica, GameEngine, Arena, Navicella, Alieno, Proiettile, Barriera e UFO non cambieranno di una riga. Si riscriveranno solo Campo e Tastiera.

### Perché GameEngine conosce tutto?

GameEngine possiede Arena, la Navicella, gli Alieni, i Proiettili, le Barriere e l'UFO ed è l'unico a gestire il ciclo di aggiornamento e il rilevamento delle collisioni. Questo lo rende una classe con molte responsabilità.

È una scelta consapevole per questa fase del progetto: permette di avere un unico punto di controllo facile da seguire e da debuggare. In Space Invaders le collisioni coinvolgono coppie di entità che appartengono ad array diversi — solo chi li possiede tutti può verificarle in modo sistematico. Quando il progetto crescerà, si potrà estrarre da GameEngine una classe CollisionManager dedicata.

# 1 Classe — Il Progetto

*Saper distinguere tra il modello astratto e l'entità concreta*

Una classe è lo stampo, non l'oggetto. Alieno è la classe; il nemico di tipo B che si trova in seconda riga, quarta colonna all'inizio del livello è l'oggetto (l'istanza). Quando progetti, devi sempre chiederti: "Sto descrivendo una categoria di cose, o una cosa specifica?"

#### Domanda 1 [Classe]

Nel modello proposto esiste un'unica classe Alieno con un attributo tipo (A, B, C). Un altro studente propone invece di creare tre classi separate: AlienoA, AlienoB, AlienoC. Dal punto di vista della progettazione OOP questa scelta è corretta? Cosa si perde e cosa si potrebbe fare invece per gestire i comportamenti diversi legati al tipo?
***Traccia:*** *Cosa hanno in comune i tre tipi di alieno (posizione, velocità, collisione)? Cosa li differenzia? La differenza sta nella struttura o solo nei valori degli attributi (sprite, punti)?*

#### Domanda 2 [Classe]

Nel modello non compare una classe chiamata Vettore che permette di rappresentare sia le coordinate della posizione che quelle della velocità. Secondo te andrebbe inserita trasferendo le coordinate da OggSpaziale? Descrivi se la sua assenza è un problema concreto per gestire, ad esempio, il movimento della formazione aliena o i proiettili.
***Traccia:*** *Il movimento della formazione aliena richiede di aggiornare la posizione di tutti gli alieni in modo coerente. Anche in OggSpaziale la posizione è centralizzata, ma quali vantaggi aggiunge una classe Vettore2D?*

#### Domanda 3 [Classe]

Un tuo compagno sostiene che Navicella e Alieno sono la stessa cosa e che basta una sola classe EntitàMobile. Ha ragione? Elenca almeno due differenze di stato o di comportamento che giustificano due classi distinte.
***Traccia:*** *Guarda il modello: la Navicella ha vite e invulnerabile, l'Alieno ha tipo e si muove in formazione. Pensa anche ai metodi: quali azioni può fare solo la Navicella del giocatore? Quali solo l'Alieno?*

# 2 Attributo — Lo Stato

*Saper identificare quali dati definiscono l'identità di un oggetto*

Gli attributi definiscono cosa un oggetto sa di se stesso. La domanda giusta non è 'dove posso mettere questo dato?' ma 'a chi appartiene questa informazione per natura?'

### Domanda 4 [Attributo]

La classe Navicella ha un attributo invulnerabile di tipo bool. A quale evento del gioco corrisponde (quando diventa true e quando torna false)? Scrivi la dichiarazione C++ dell'attributo e spiega perché è utile avere questo stato separato rispetto a gestirlo solo tramite le vite.
***Traccia:*** *Dopo aver perso una vita, la navicella ricompare al centro in basso: se fosse subito vulnerabile, verrebbe colpita immediatamente dai proiettili nemici vicini. Quanto dura l'invulnerabilità? Chi decide quando termina?*

### Domanda 5 [Attributo]

La classe OggSpaziale ha sia posizione che velocità come Vettore2D. In Space Invaders la navicella si muove solo quando il giocatore preme un tasto (non ha inerzia). Allora perché mantenere l'attributo velocità? Non basterebbe aggiornare la posizione direttamente senza memorizzarla?
***Traccia:*** *Pensa agli altri oggetti che usano OggSpaziale: i proiettili e l'UFO si muovono a velocità costante autonomamente. Se OggSpaziale non avesse velocità, come gestiresti il loro aggiornamento? Vorresti duplicare la logica in ogni classe?*

### Domanda 6 [Attributo]

La classe Alieno ha un attributo tipo di tipo char ('A', 'B', 'C'). Un altro studente propone di usare un intero: 1, 2, 3. Ci sono vantaggi e svantaggi in entrambe le scelte? Quale preferiresti nel codice C++ del gioco e perché?
***Traccia:*** *Cosa succede quando devi calcolare i punti in base al tipo? Con quale tipo è più semplice fare un confronto o un calcolo? E se un giorno volessi aggiungere un tipo D?*

### Domanda 7 [Attributo]

La classe Proiettile ha un attributo tempoVita di tipo int. A cosa serve? Dove viene decrementato? Cosa succede se non lo avessimo e i proiettili non scadessero mai?
***Traccia:*** *In Space Invaders il proiettile del giocatore deve scomparire quando raggiunge il bordo superiore. Senza tempoVita, come sapresti quando rimuoverlo? E se un proiettile nemico uscisse dai bordi laterali, cosa potrebbe succedere?*

# 3 Metodo — Il Comportamento

*Valutare la capacità di ripartire le azioni in blocchi logici*

Un metodo è una responsabilità ben definita. Deve fare una cosa sola, farla bene, e il suo nome deve dire esattamente cosa fa. Se per spiegare un metodo usi la parola 'e', probabilmente dovrebbe essere diviso in due.

### Domanda 8 [Metodo]

Descrivi il comportamento del metodo aggiorna() nella classe Alieno. Cosa deve fare in più rispetto al metodo aggiorna() di OggSpaziale? In particolare: come gestisce il cambio di direzione quando la formazione raggiunge il bordo? Chi decide quando tutta la formazione deve invertire la direzione?
***Traccia:*** *aggiorna() di OggSpaziale calcola la nuova posizione in base alla velocità. aggiorna() di Alieno deve fare lo stesso, ma anche rilevare se ha raggiunto il bordo. Chi conosce la posizione di tutti gli alieni per decidere se la formazione intera deve scendere? Forse GameEngine?*

### Domanda 9 [Metodo]

Il metodo aggiorna() compare in OggSpaziale, in Alieno, in Proiettile e in UFO. Tutti e quattro fanno la stessa cosa? Descrivi cosa fa aggiorna() per almeno due di queste classi e spiega perché ha senso che ogni classe abbia la propria versione del metodo.
***Traccia:*** *aggiorna() di OggSpaziale calcola la nuova posizione in base alla velocità. aggiorna() di Proiettile fa lo stesso ma in più decrementa tempoVita. aggiorna() di UFO può includere anche la logica di movimento orizzontale autonomo. Cosa cambia?*

### Domanda 10 [Metodo]

Un metodo chiamato gestisciTurno() in una ipotetica classe GestoreGioco fa queste cose: aggiorna tutti gli oggetti, controlla tutte le collisioni, rimuove gli oggetti non attivi, fa sparare casualmente gli alieni, aggiorna il punteggio e ridisegna lo schermo. È un buon metodo? Identificane i problemi e proponi come suddividerlo.
***Traccia:*** *Conta quante responsabilità diverse ha questo metodo. Quali sono logica di gioco e quali sono logica di visualizzazione? Quali potrebbero diventare metodi privati separati chiamati da gestisciTurno()?*

### Domanda 11 [Metodo]

Il metodo collideWith(o:OggSpaziale) di OggSpaziale controlla se due oggetti si sovrappongono usando i loro raggi. Questo metodo dovrebbe stare in OggSpaziale oppure in una classe esterna GestoreCollisioni? Confronta le due scelte.
***Traccia:*** *Se sta in OggSpaziale, l'oggetto sa da solo se è stato colpito. Se sta in GestoreCollisioni, nessun oggetto conosce gli altri. Quale delle due rende più facile aggiungere nuovi tipi di oggetti in futuro?*

# 4 Incapsulamento — Protezione e Interfaccia

*Verificare se si sa proteggere l'integrità dei dati*

Incapsulare non significa solo mettere private davanti a tutto. Significa decidere consapevolmente quale parte del tuo oggetto è visibile all'esterno (l'interfaccia pubblica) e quale è un dettaglio interno che nessun altro deve toccare direttamente.

### Domanda 12 [Incapsulamento]

Nella classe OggSpaziale, posizione e velocità sono private. Un tuo compagno dice: 'basterebbe renderle public, così il codice del gioco può aggiornare posizione e velocità direttamente senza passare per i metodi'. Spiega almeno due problemi concreti che nascerebbero in Space Invaders se si seguisse questo consiglio.
***Traccia:*** *Se posizione fosse public, chi garantisce che un alieno non venga spostato fuori dai limiti dell'Arena? E se due parti del codice modificassero la posizione contemporaneamente in modo incoerente, cosa succederebbe alla formazione?*

### Domanda 13 [Incapsulamento]

La classe Navicella espone il metodo perdiVita() come pubblico. Questo significa che chiunque può far perdere una vita alla navicella in qualsiasi momento. È corretto? Chi dovrebbe avere il diritto di chiamare questo metodo, e come progetteresti l'interfaccia per limitarne l'uso?
***Traccia:*** *Solo una collisione con un proiettile nemico o con un alieno dovrebbe causare la perdita di una vita. Ma in C++ non puoi limitare 'chi' chiama un metodo pubblico. Come si gestisce convenzionalmente questo problema di progettazione?*

### ⚠ Domanda 14 trabocchetto [Incapsulamento]

La classe Barriera ha il metodo assorbi() che riduce i propri punti vita di uno. Se rendessimo pubblico l'attributo puntiVita (invece di avere getPuntiVita()), il codice esterno potrebbe modificarlo direttamente — ad esempio scrivere barriera.puntiVita = 0. Quali conseguenze avrebbe nel gioco?
***Traccia:*** *Cosa succede se puntiVita diventa negativo? Il metodo assorbi() usa puntiVita per decidere se la barriera è ancora attiva: cosa restituisce se il valore è stato impostato a -5 dall'esterno? L'oggetto diventerebbe inconsistente.*

# 5 Composizione — Relazione "Ha un"

*Valutare la capacità di costruire oggetti complessi assemblando pezzi semplici*

La composizione è il modo principale in cui gli oggetti collaborano: un oggetto contiene un altro oggetto come attributo. Navicella HA UN OggSpaziale. Alieno HA UN OggSpaziale. Proiettile HA UN OggSpaziale. Costruire bene la composizione significa evitare oggetti troppo grandi e responsabilità duplicate.

### Domanda 15 [Composizione]

Navicella, Alieno, Proiettile, Barriera e UFO contengono tutti un attributo OggSpaziale. Questo significa che la logica di posizione, velocità e collisione è duplicata cinque volte? Oppure è la scelta corretta? Giustifica la tua risposta usando il concetto di composizione.
***Traccia:*** *Duplicare codice e duplicare la struttura sono due cose diverse. Tutte e cinque le classi usano OggSpaziale, ma chi implementa la logica di movimento e collisione? Quante volte è scritta quella logica nel codice?*

### Domanda 16 [Composizione]

Se volessi aggiungere un power-up (es. un Cannone Rapido che appare sullo schermo e che la navicella può raccogliere), progetteresti una nuova classe CannoneRapido con un

proprio OggSpaziale, oppure riutilizzeresti una classe esistente aggiungendo attributi? Motiva la scelta.
***Traccia:*** *Il CannoneRapido ha una posizione, una velocità (magari nulla), un raggio di collisione e uno stato attivo/non attivo. Esiste già nel modello una classe con esattamente queste caratteristiche? Cosa cambia aggiungendo solo il comportamento specifico?*

### ⚠ Domanda 17 trabocchetto [Composizione]

La classe Vettore2D è usata sia per posizione che per velocità in OggSpaziale. Uno studente dice: 'basterebbe usare quattro double separati: posX, posY, velX, velY, senza creare una classe Vettore2D'. Quali vantaggi perde con questa scelta?
***Traccia:*** *Il metodo aggiorna() calcola posizione = posizione + velocità. Con Vettore2D questa operazione è una riga. Con quattro double separati, quante righe diventano? E se aggiungi la rotazione dell'UFO, che richiede operazioni su entrambe le componenti?*

# 6 Responsabilità — Chi fa cosa

*Capire se si sa evitare la creazione di 'Classi Dio' che fanno tutto*

Ogni classe deve avere una sola ragione di esistere e una responsabilità chiara. Se una classe conosce troppo degli altri, o fa cose che non le competono, diventa fragile: ogni modifica al gioco richiede di toccarla. La domanda da farsi sempre è: 'Questa azione appartiene davvero a questa classe?'

### Domanda 18 [Responsabilità]

Immagina che uno studente crei una sola classe SpaceInvadersGame che gestisce tutto: gli oggetti spaziali, le collisioni, il punteggio, la logica della formazione aliena, la grafica e l'input da tastiera. Quali sono i problemi pratici di questa scelta quando il gioco deve essere modificato (es. aggiungere un nuovo tipo di alieno o cambiare la logica di discesa)?
***Traccia:*** *Prova a pensare a cosa devi toccare nella classe SpaceInvadersGame ogni volta che cambia anche solo una piccola regola. E se vuoi riusare la logica degli alieni in un gioco diverso?*

### Domanda 19 [Responsabilità]

La logica di movimento della formazione (tutti gli alieni si spostano nella stessa direzione e scendono quando raggiungono il bordo) potrebbe stare nel metodo aggiorna() di ciascun Alieno oppure in una classe esterna GestoreFormazione. Quale delle due posizioni ti sembra più corretta e perché? Ci sono argomenti validi per entrambe?
***Traccia:*** *Chi possiede la regola di formazione? È una caratteristica intrinseca di ogni alieno (sa come muoversi in gruppo) o è una regola esterna del gioco che coordina tutti gli alieni dall'esterno? Quale delle due classi conosce la posizione di tutti gli alieni?*

### ⚠ Domanda 20 trabocchetto [Responsabilità]

Il metodo aggiorna() di OggSpaziale aggiorna la posizione in base alla velocità. Dovrebbe anche occuparsi del controllo dei bordi (se esci a destra ti fermi)? Oppure questa responsabilità appartiene a qualcun altro? Proponi una soluzione e discutine i pro e i contro.
***Traccia:*** *Se il controllo bordi è dentro OggSpaziale, ogni oggetto gestisce autonomamente i propri confini. Ma OggSpaziale dovrebbe conoscere le dimensioni dello schermo di gioco. È corretto che un oggetto fisico conosca i limiti del campo da gioco?*

### Domanda 21 [Responsabilità]

Scrivi la dichiarazione C++ (solo l'intestazione, non l'implementazione) di una classe GestoreOndata che abbia la responsabilità di creare e gestire le ondate di alieni per ogni livello. Quali attributi deve avere? Quali metodi? Giustifica ogni scelta.
***Traccia:*** *Una classe GestoreOndata non deve implementare la logica interna degli alieni: deve coordinarli. Quanti alieni ci sono in ogni ondata? Come aumenta la difficoltà tra un livello e l'altro? Chi la chiama e quando?*

# Riepilogo delle competenze

| N. | Competenza |
|---|---|
| 1 Classe | Saper distinguere il modello astratto dall'entità concreta |
| 2 Attributo | Identificare i dati che definiscono l'identità di un oggetto |
| 3 Metodo | Ripartire le azioni in blocchi logici con responsabilità chiara |
| 4 Incapsulamento | Proteggere l'integrità dei dati e progettare l'interfaccia |
| 5 Composizione | Costruire oggetti complessi assemblando pezzi semplici |
| 6 Responsabilità | Distribuire bene le responsabilità, evitare le Classi Dio |